PlannerDNA: Learning and Blind-Testing a Personal Pairwise Preference Model from Explicit Design Choices in a Synthetic 3D Urban-Planning Sandbox

Alpha
Independent Researcher • Technical report • October 2026

Abstract. Planning students are routinely asked to balance competing objectives, yet the trade-offs they actually favour are rarely made explicit, let alone tested. We present PlannerDNA, a local, single-user 3D sandbox in which a participant edits a procedurally generated 20 × 20 neighbourhood, saves two complete alternatives and records an explicit preference between them. Five transparent outcome metrics (park access within a 400 m road-network walk, housing capacity, park share, intervention cost and lost original housing) are recomputed deterministically after every edit and independently re-verified by a local server. On request, a pairwise logistic (Bradley–Terry) model with mirrored rows, no intercept and L2 regularisation is fitted to the participant's own feasible A/B choices, with scenario-level bootstrap intervals and leave-one-scenario-out diagnostics against simple decision rules. Predictive validity is then assessed by a pre-registered, blinded protocol: the model is frozen by checksum, test scenarios are verified unseen, every prediction is committed with a SHA-256 seal before the choice, screen sides are randomised from a server-held seed, repeated pairs measure the participant's own consistency, and the report is stored as hashed canonical JSON that is recomputed on every read. In a single-participant case study (30 training comparisons across 5 synthetic cities, 20 of them generated by a fixed rule), the frozen model predicted 8 of 9 scored held-out choices (88.9%; 95% scenario-bootstrap interval 50%–100%), exactly matching the rule “always choose more park access”, with which it agreed on every scored pair. We argue that the contribution is the workflow (explicit trade-offs, explicit labels and an evaluation that cannot be flattered) rather than the accuracy figure.

Keywords: preference learning; paired comparisons; Bradley–Terry model; planning support systems; serious games; blinded evaluation; pre-registration; reproducibility; green-space accessibility.

1 Introduction

Urban planning is, to a large degree, the negotiation of trade-offs. Adding a park can bring more homes within walking distance of green space, which a substantial body of evidence associates with health and well-being [1]; but the same land might otherwise hold housing, and every intervention has a cost and may displace existing residents. Planning support systems and serious games have long been proposed to make such trade-offs tangible for professionals, students and the public [2, 3]. What they rarely do is record, explicitly and verifiably, which trade-offs a particular person actually prefers, and then test whether a summary of those preferences predicts that person's later choices.

Learning from pairwise comparisons is a mature field, from Thurstone's law of comparative judgment and the Bradley–Terry and Luce models to conditional logit choice models [4–7], and from pairwise ranking in machine learning to preference-based reinforcement learning [8–11]. With few choices, however, fitted preference weights are easily over-interpreted, and evaluations are easily flattered: by leakage between training and test data [12, 13], by selecting test items that suit the model, by showing predictions that anchor the participant [14], or by choosing the analysis after seeing the data [15].

This paper describes PlannerDNA, a system that couples a deterministic urban-design sandbox with an explicit labelling regime, a deliberately simple and interpretable preference model, and a blinded, pre-registered evaluation protocol whose integrity is enforced by the software rather than by convention. Its contributions are:

- a deterministic, verifiable synthetic planning environment with five transparent proxy metrics, implemented in C# and re-implemented in Python, with every stored design recomputed by the server (Section 4);

- an explicit-choice labelling regime in which only feasible A/B choices made in training mode can become training data, and interaction traces never do (Section 5);
- a symmetric pairwise logistic model with readiness refusals, scenario-level bootstrap uncertainty and leave-one-scenario-out diagnostics against rules of thumb (Section 6);
- a blinded held-out evaluation protocol using cryptographic commitments, server-held randomisation, model-blind pair selection and repeat-consistency measurement, with a self-verifying report (Section 7);
- a single-participant case study, reported together with the reasons it does not show more than a one-line rule (Section 9, Section 10).

2 Related work

2.1 Planning support, serious games and accessibility measures

Planning support systems combine models, data and visualisation to inform planning tasks [2], and serious games have been used to make participation more engaging and to expose trade-offs to non-experts [3]. Accessibility to green space is commonly operationalised as the share of residents within a distance threshold of a park, with network (street) distance preferred over straight-line buffers because it reflects the routes people can actually walk [16]. PlannerDNA adopts a road-network distance with a 400 m threshold, but its metrics are deliberately simplified proxies on a synthetic grid and are not calibrated to any real place.

2.2 Paired comparisons and preference learning

The Bradley–Terry model expresses the probability that one item is preferred to another as a logistic function of a difference in latent utilities [5]; Thurstone's earlier formulation uses a normal link [4], and Luce's choice axiom generalises it to choice sets [6]. When utilities are linear in observed attributes, the model becomes a conditional logit, the workhorse of discrete-choice and stated-choice analysis [7, 17]. In machine learning, the same “pairwise transform” (learning a linear classifier on attribute differences) underlies ranking SVMs [8, 9] and the broader preference learning literature [10], and pairwise human judgments now train large reward models [11]. PlannerDNA's labels are *stated* choices in a hypothetical setting, not revealed preferences in the economic sense [18], and human choices can be inconsistent or intransitive [19], which is why the protocol measures repeat consistency.

2.3 Interpretable models and evaluation integrity

For decisions that people must understand and contest, inherently interpretable models are preferable to post hoc explanations of opaque ones [20]; a ridge-penalised linear model [21] on five named outcomes is about as interpretable as a predictive model can be. On the evaluation side, leakage is a leading cause of irreproducible machine-learning results [12, 13]; grouped (blocked) cross-validation is recommended when observations are clustered [22], and the cluster bootstrap resamples whole groups for the same reason [23–25]. Pre-registration separates confirmatory from exploratory analysis [15]. Probabilistic predictions are scored with proper scoring rules such as the Brier score and log loss [26, 27], and calibration is assessed separately [28, 29]. Cryptographic hash functions [30] allow a prediction to be committed before an outcome is known and verified afterwards. Finally, the local-first software movement argues for applications that keep working, and keep the user's data, without a cloud service [31].

3 System overview

PlannerDNA consists of a Unity 6 client [32] and an optional local backend written in Python with FastAPI and SQLite [33, 34] (Fig. 1). Both run on one computer and communicate only over the loopback interface; there are no cloud services and no telemetry. The client's core (city state, generation, validation, metrics, cost, constraints, history and storage formats) is plain C# with no engine dependencies, so it is covered by fast unit tests. The city state is immutable: every accepted edit produces a new state, which makes undo and redo exact snapshot restores.

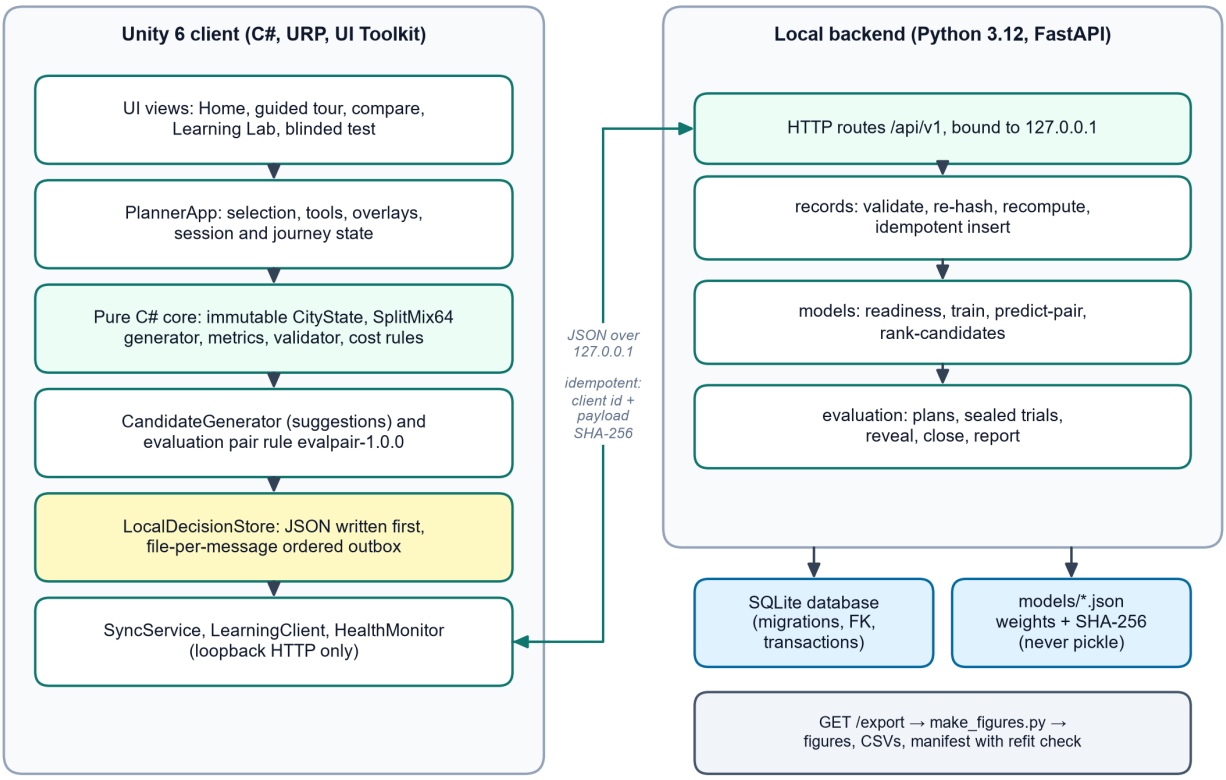


Fig. 1. Architecture. The Unity client writes every record locally first and delivers it, in order, through a file-per-message outbox; the backend re-hashes and recomputes every design before storing it. Model files are JSON with a SHA-256 checksum. Figures are generated only from an export.

The client is offline-first in the sense of [31]: the sandbox never waits for the server. Records are delivered strictly in order (so a decision can never arrive before the designs it refers to) and idempotently: each carries a client id and the SHA-256 of its payload, so a resend after a lost acknowledgement is a harmless duplicate, while reuse of an id with different content is refused.

A session follows four stages shown on a journey line at the top of the screen (Fig. 2): **Explore** the generated neighbourhood; **Design A and B** from the same baseline; **Compare and choose**; and **Learn**, where the participant may train a model, inspect it, request ranked suggestions, or start a blinded test. A Home screen and a 13-step guided tour, written for first-year urban-planning undergraduates, spotlight each part of the interface and wait for the participant to perform the real action.

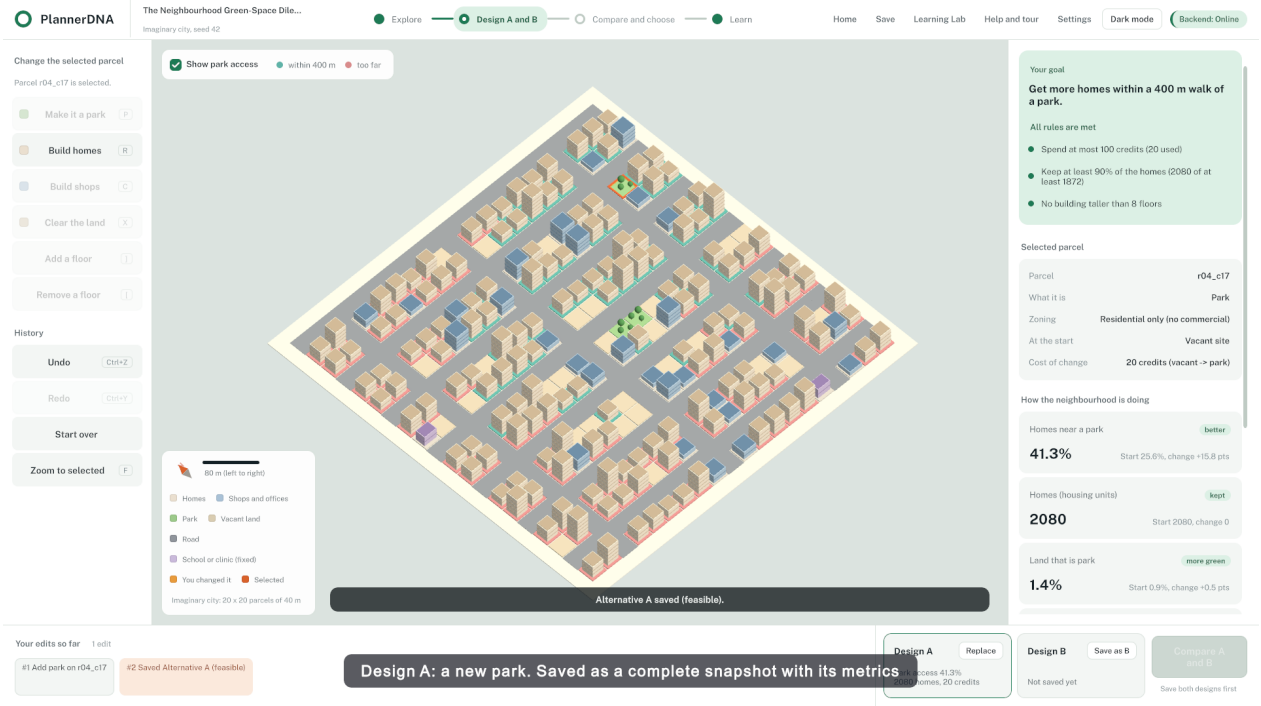


Fig. 2. The sandbox. Left: tools and history. Centre: the 3D city with the park-access overlay (teal homes reach a park within 400 m by road; rose homes do not). Right: the mission, hard rules as green or red dots, the selected parcel and live metrics. Bottom: saved alternatives.

4 Synthetic planning environment

The exercise is the *Neighbourhood Green-Space Dilemma*: increase the share of housing capacity within a 400 m road-network walk of a park while keeping housing capacity at or above 90% of the baseline, spending at most 100 synthetic credits and never exceeding eight floors. The mission text is generated from the scenario parameters (Table 1), so text and rules cannot drift apart.

Table 1. Scenario parameters (generator green-gen-1.0.0, metrics metrics-1.0.0, cost rules cost-rules-1.0.0).

Parameter	Value
Grid	20 × 20 parcels of 40 m (800 m × 800 m)
Budget	100 synthetic credits
Minimum housing	ceil(0.9 × baseline units), integer arithmetic
Maximum height	8 floors
Access threshold	400 m along roads
Units per floor; floors of a new building	4; 2

4.1 Deterministic generation

A seed fully determines the city. Generation uses SplitMix64 [35] rather than a platform random-number generator because its output is specified and identical across runtimes; a unit test pins reference values. Every third row is a road and three vertical roads are drawn from three bands, so every parcel touches a road. About 10% of parcels are vacant, commercial parcels concentrate beside the middle vertical road, residential parcels have two to six floors (35% zoned residential-only), one or two small parks sit in one quadrant, and one or two civic parcels are protected. A candidate city is accepted only if the road network is connected, baseline access is below 75%, there are at least three vacant parcels and at least one feasible vacant-to-park edit improves access; otherwise generation retries, up to 32 attempts, and then reports an error instead of producing a broken city.

4.2 Metrics

Housing capacity is the sum over residential parcels of floors × 4 units, a capacity proxy rather than a population. Park access uses a walk distance along roads only (Fig. 3): a multi-source breadth-first search starts from every road cell adjacent to a park, and a residential parcel uses its best adjacent road cell. With k road steps between the two road cells, the walk distance and access share are

$$d_i = 40 + 40\,k_i + 40 \text{ m}, \qquad i \text{ is served} \Leftrightarrow d_i \le 400 \text{ m} \tag{1}$$

$$\text{Access} = 100 \cdot \frac{\sum_{i \in R} u_i \,\mathbf{1}[d_i \le 400]}{\sum_{i \in R} u_i}, \qquad u_i = 4\,\text{floors}_i \tag{2}$$

where R is the set of residential parcels; access is reported as not applicable when housing is zero. With 40 m cells the threshold falls between 400 m (served) and the next possible distance, 440 m (not served). Park share is the number of park parcels over non-road parcels. Intervention cost compares each parcel's baseline state b_i with its current state s_i:

$$\text{Cost} = \sum_i c(b_i, s_i), \qquad c(b, b) = 0 \tag{3}$$

using the table in Table 2. Because cost depends only on the difference between baseline and current state, restoring a parcel always costs nothing and the order of edits cannot game the budget. Lost original housing sums, over parcels that were residential at baseline, the units removed; it is a displacement proxy reported separately from net housing change.

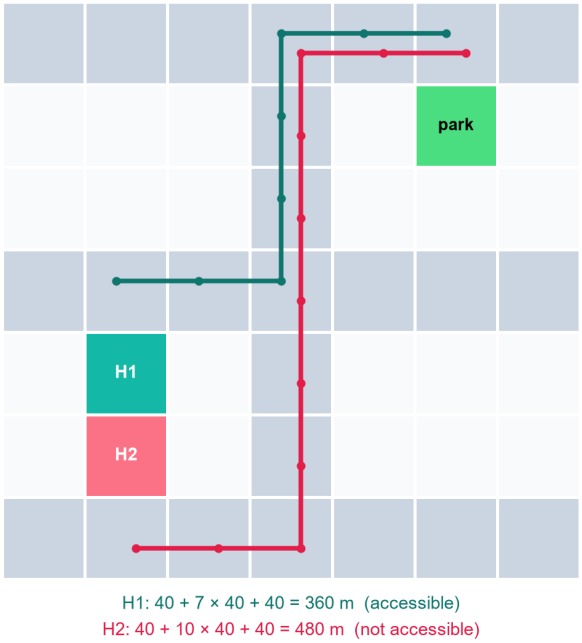


Fig. 3. Road-network walk distance. Only road cells (grey) are walkable, with no diagonal moves or shortcuts through parcels. H1 reaches the park in 360 m and is served; H2, one parcel away, needs 480 m and is not.

Table 2. Cost rules (cost-rules-1.0.0); costs compare baseline and current parcel states.

Baseline → current	Credits
unchanged	0
vacant → park	20
vacant or park → new residential or commercial	25 + 5 × floors (35 for 2 floors)
residential or commercial → park	35
residential or commercial → vacant (demolish)	15
park → vacant	10
residential ↔ commercial	20, + 5 per added floor or + 3 per removed floor
same building use, floors changed	+ 5 per added floor, + 3 per removed floor
anything involving a road or civic parcel	undefined: the edit is refused

4.3 Constraints, validation and cross-implementation checks

Four hard constraints are evaluated after every edit: budget (spent ≤ 100), housing floor, maximum floors, and a defined cost rule for every changed parcel. A validator refuses edits to roads and civic parcels, parks on parks, commercial uses on residential-only land and heights outside one to eight floors, each with a plain-language reason; refused attempts are counted but never enter the design timeline. Infeasible designs may be explored and saved, but are marked as such.

The metric engine exists twice: in C# for the interactive client and in Python for the server. Both compute the same FNV-1a content hash of a city. Contract fixtures exported from Unity are replayed by the Python test suite, and at run time the server rebuilds every submitted design from its snapshot, recomputes hash, metrics, constraints and features, and refuses any disagreement. The database therefore cannot hold a design whose numbers disagree with the published rules.

5 Data collection and labelling

The unit of analysis is one explicit comparison between two saved designs. Edits, undo clicks, refused attempts and dwell time are logged for auditability but are never used as labels (Fig. 4). A comparison is **training-eligible** only if the choice is A or B (ties and skips are stored but not used), it was made in training mode (answers given during a blinded test are never training data, not even after the test closes), both designs are feasible, both come from the same registered scenario and baseline, and both carry the same feature version. Unity and the server both check these rules; the server's result is authoritative.

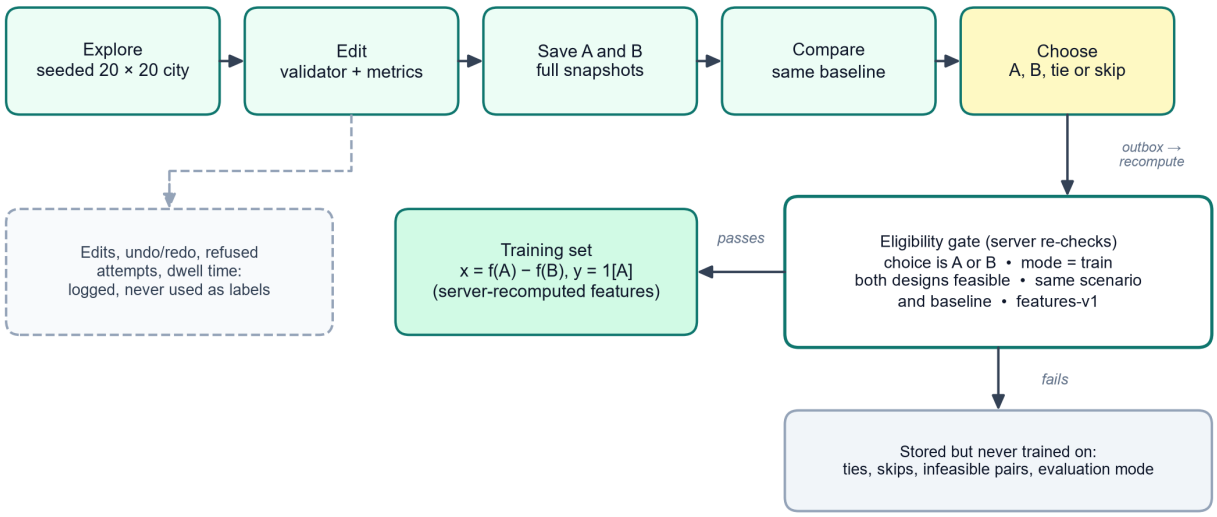


Fig. 4. From interaction to label. Only an explicit A-or-B choice between two feasible designs made in training mode becomes a training example; everything else is stored for transparency but never trained on.

Each design D is summarised by five features (features-v1), each a difference from the scenario baseline and oriented so that higher means "more of a commonly desirable outcome" (Table 3). The orientation fixes only the sign convention; it is not a preference weight, and no weights exist until a model is trained on the participant's own choices.

Table 3. Feature vector f(D) (features-v1). H_0 is baseline housing capacity; all values are fractions.

Feature	Definition	Higher means
delta_access_fraction	(access % − baseline access %) / 100	more homes within a 400 m walk of a park
delta_housing_fraction	(units − H_0) / H_0	more housing capacity
delta_park_share_fraction	(park % − baseline park %) / 100	more park land
negative_budget_fraction	− spent credits / 100	lower cost
negative_lost_housing_fraction	− lost original units / H_0	fewer original homes removed

The model input for a comparison is the difference x = f(A) − f(B) with label y = 1 if A was chosen. Comparisons whose two designs have identical measured outcomes (x = 0) carry no information about trade-offs; they are counted, reported as ignored and never fitted.

6 Preference model

6.1 Specification and symmetry

The model is a linear Bradley–Terry, or conditional logit, model on outcome differences [5, 7]:

$$P(A \succ B \mid x) = \sigma(w^\top x) = \frac{1}{1 + e^{-w^\top x}}, \qquad x = f(A) - f(B) \tag{4}$$

There is no intercept. An intercept would absorb any tendency to choose whichever design is presented as A, i.e. a screen-position habit, and would make the model's predictions depend on presentation order. Without it, and because σ(−z) = 1 − σ(z),

$$P(B \succ A) = \sigma(-w^\top x) = 1 - \sigma(w^\top x) = 1 - P(A \succ B) \tag{5}$$

holds exactly. To make the fitted weights themselves independent of which design was labelled A, every comparison is added twice, as (x, y) and as (−x, 1 − y). The mirrored copies balance the fit; they are not additional evidence, and every count reported anywhere is of unique comparisons.

6.2 Estimation

Features are scaled by their root-mean-square over the 2n mirrored training rows. Because the mirrored rows have mean exactly zero, the standard deviation computed by a non-centring StandardScaler equals the RMS, and not centring preserves the symmetry. With every difference divided by its feature scale s_j, the standardised weights solve an L2-penalised logistic regression [21]:

$$\hat{w}^{(s)} = \arg\min_w \ \frac{1}{2}\|w\|^2 + C\sum_{r=1}^{2n} \ell\big(y_r,\ \sigma(w^\top \tilde{x}_r)\big), \qquad s_j = \sqrt{\frac{1}{2n}\sum_{r=1}^{2n} x_{rj}^2} \tag{6}$$

where ℓ is the logistic loss. The implementation uses scikit-learn's LogisticRegression [36] with C = 1, the lbfgs solver, a tolerance of 10^{-8}, at most 1000 iterations and a fixed random seed (20261025), on NumPy and SciPy [37, 38]. A standardised weight is the change in log-odds for one typical (RMS) training difference in that feature, so standardised weights are comparable across features; raw weights $w_j = w^{(s)}_j / s_j$ are used for utilities and probabilities. A feature that never varied in training receives weight 0 and is reported as *not estimated* rather than unimportant, and |standardised weight| below 0.1 (odds change of less than about 10%) is reported as little or no association. The utility of a design, $w \cdot f(D)$, is unit-free and meaningful only for ordering designs within a scenario.

6.3 Readiness

Training refuses rather than guessing. It requires at least 10 informative comparisons from at least two scenarios, and the RMS-scaled difference matrix must have rank at least 2, so that the data span at least two independent trade-off directions; otherwise the server returns HTTP 422 with the reason (not enough comparisons, a single scenario, or homogeneous comparisons) and writes nothing. Below 30 comparisons over five scenarios the model is labelled *exploratory* wherever it appears. These thresholds are heuristics, not statistical adequacy guarantees.

6.4 Uncertainty and diagnostics

Choices within one scenario share a city and are not independent, so uncertainty is estimated by a scenario-level (cluster) bootstrap [24, 25]: scenarios are resampled with replacement 200 times with the feature scale held fixed, the model is refitted, and each standardised weight gets a 90% percentile interval and the share of resamples whose sign agrees with the full fit. A weight whose interval includes zero is reported as directionally uncertain.

Leave-one-scenario-out (LOSO) cross-validation [22] predicts each scenario's comparisons with a model and scaler fitted without that scenario. The same comparisons are scored by four transparent baselines: chance (always 0.5); always the cheaper design; always the design with more park access; and the sign of the equal-weight sum of the five features. Deterministic rules predict 1, 0 or 0.5 (for ties). With predictions p_i and outcomes y_i, accuracy counts an undecided prediction as half correct, and the Brier score [26] and log loss are the usual proper scoring rules [27]:

$$\mathrm{Acc} = \frac{1}{n}\sum_{i=1}^{n}\left(\mathbf{1}\left[(p_i - \tfrac{1}{2})(y_i - \tfrac{1}{2}) > 0\right] + \tfrac{1}{2}\,\mathbf{1}\left[p_i = \tfrac{1}{2}\right]\right) \tag{7}$$

$$\mathrm{BS} = \frac{1}{n}\sum_i (p_i - y_i)^2, \qquad \mathrm{LL} = -\frac{1}{n}\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)] \tag{8}$$

Log loss is reported only for probabilistic predictors. Training-set accuracy is also shown, labelled optimistic, with a separability warning at 100%. LOSO is a diagnostic inside the training data and is never described as a held-out test; only a closed blinded evaluation (Section 7) is.

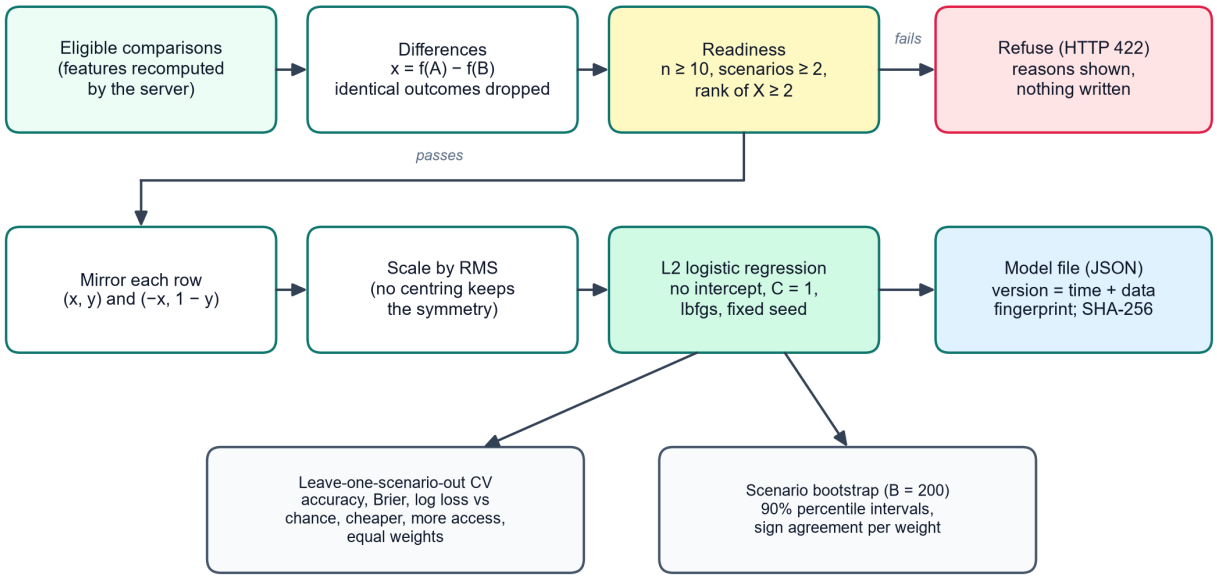


Fig. 5. Model fitting. Readiness checks gate training; mirrored, RMS-scaled differences are fitted by an L2 logistic regression without intercept; the result is a checksummed JSON file. Diagnostics are computed alongside and shown with the weights.

6.5 Training, versioning and use

Training happens only when the participant presses **Train my model**, and only once every local record has been delivered, so a model always reflects every choice made so far. The model version is *pm-<UTC timestamp>-<fingerprint>*, where the fingerprint is the SHA-256 of the canonical JSON of the ML and feature versions, hyperparameters and every training row; retraining on unchanged data returns the existing model. Each model is stored as a JSON file (never pickle, so loading cannot execute code) containing feature names, raw and standardised weights, the scale, hyperparameters, seed, training comparison and scenario ids, counts, fingerprint and package versions. Its SHA-256 is stored in the database, and every prediction reloads the file and refuses if the hash differs.

A trained model is used in three ways. In **Compare**, its prediction for the pair is requested only after the participant has confirmed a choice, to avoid anchoring [14], and is shown with a note on whether that comparison was in the training data (Fig. 6). In the **Learning Lab**, weights, intervals and diagnostics are displayed with plain-language statements. For **suggestions**, Unity generates up to 30 candidate designs by applying one to four seeded, validator-approved edits to the baseline (at most 800 attempts), drops infeasible designs, exact duplicates and near-duplicates, and sends them to the server, which recomputes each candidate, refuses tampered or infeasible ones, ranks them by utility and returns the top k. Suggestions are labelled as ranked synthetic candidates, never as optimal plans.

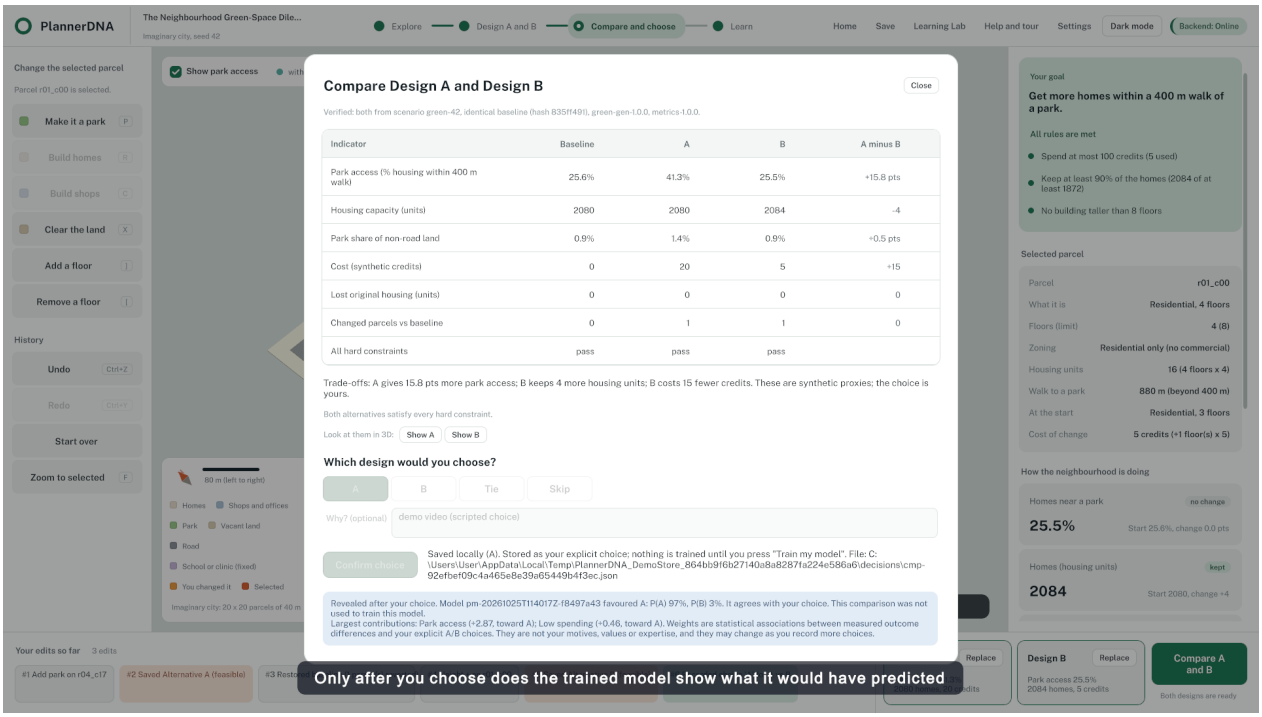


Fig. 6. Compare and choose. Both designs are tabulated against the same baseline with a plain-language trade-off summary. The participant chooses A, B, tie or skip and may give a reason; only then does the model's prediction appear.

7 Blinded held-out evaluation protocol

The evaluation asks: *does the frozen model predict choices it has never seen, in cities it was never trained on, better than simple rules of thumb?* The protocol (blinded-eval-1.0.0, Fig. 7) is designed so that neither the software nor the participant can flatter the answer, and each safeguard is enforced by the server as a refusal rather than left to good practice.

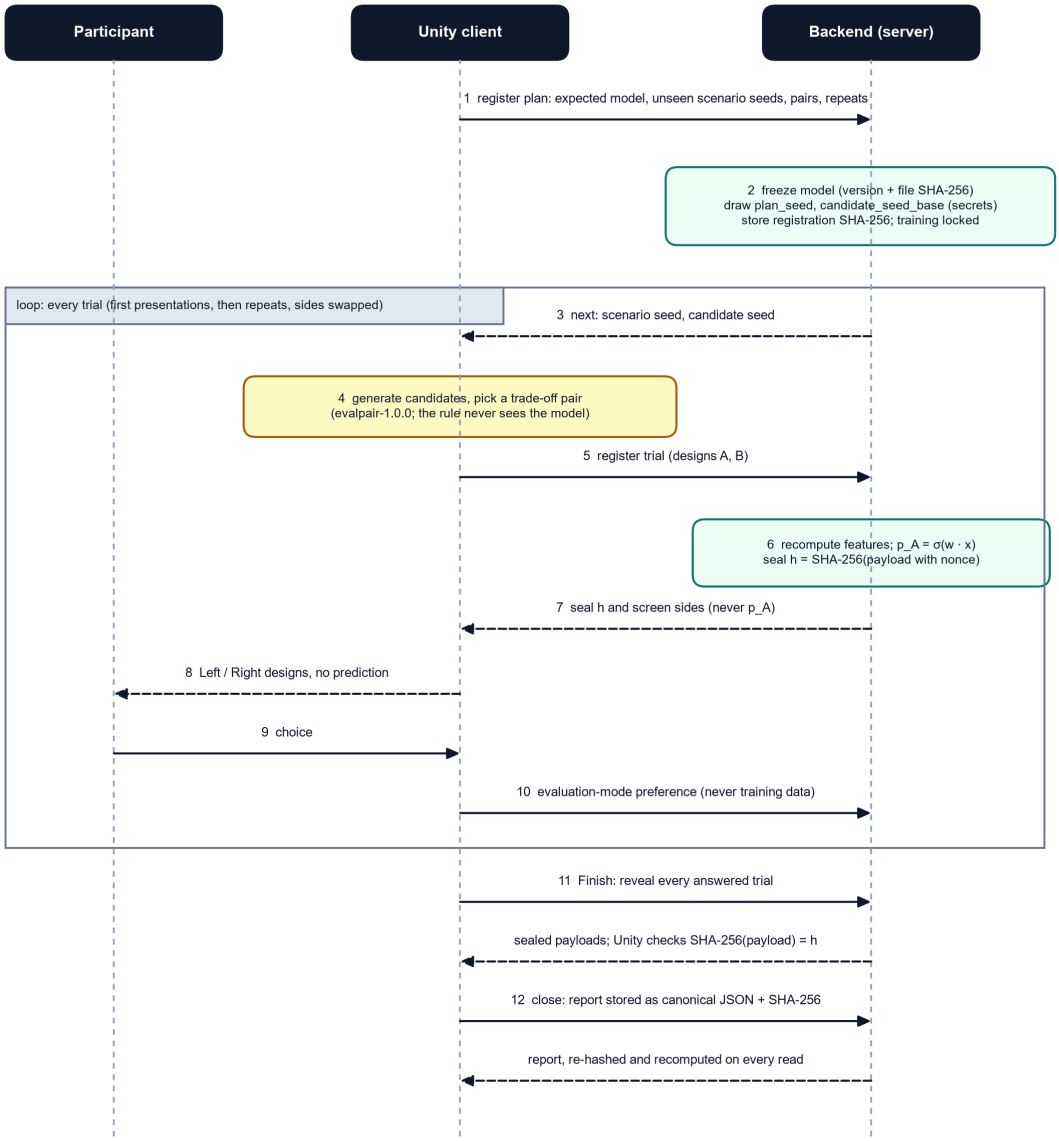


Fig. 7. Sequence of the blinded evaluation. The prediction for each trial is computed and committed before the trial is shown; the participant sees only Left and Right; all seals are opened and checked only after the last choice.

7.1 Pre-registration and freezing

The client registers a plan containing the expected model version, two to twenty evaluation scenarios (identifier and seed), one to ten pairs per scenario, a number of repeated pairs and the pair rule. The server freezes the model by storing its version and the SHA-256 of its file; every later prediction reloads the file and refuses if the hash differs. Each scenario is refused if its identifier or seed appears in the model's training data, if it already has any training-mode choice, or if an earlier plan used it; Unity draws fresh seeds between 10^5 and 10^9 and retries on refusal. The server then draws two secrets with a cryptographic random source, a plan seed (screen sides and repeat selection) and a candidate-seed base (which designs are generated), and stores the SHA-256 of the canonical registration. While a plan is open the server refuses to train any model, to accept practice choices on evaluation scenarios, to predict or rank on them, or to open a second plan (Table 4). Exports taken mid-test omit unrevealed predictions.

Table 4. Server-enforced refusals while a blinded test is open.

Attempt	Response
Train any model	409 evaluation_in_progress
Training-mode choice on an evaluation scenario	409 scenario_reserved_for_evaluation
predict-pair or rank-candidates on an evaluation scenario	409 sealed_for_evaluation
Open a second plan	409 evaluation_already_open
Scenario seen in training, practice or an earlier plan	422 scenario_not_unseen
Reveal a prediction before the choice is recorded	409 choice_not_recorded

7.2 Model-blind pair selection

For trial t the candidate seed is the secret base plus t. Unity generates candidates with the same generator used for suggestions and, using a deterministic random generator seeded by the candidate seed, picks a feasible pair that is a genuine trade-off: each design is better on at least one of the five features (if no such pair exists, any pair with different features). Because this rule (evalpair-1.0.0) never sees the model, test pairs cannot be selected to suit it. The server refuses wrong seeds, wrong rules, infeasible, identical or uninformative pairs, and designs used in any earlier comparison or trial.

7.3 Sealed predictions

On registration of a trial the server recomputes both designs' features, computes p_A with the frozen weights and stores a sealed payload together with its hash:

$$h_t = \mathrm{SHA256}(\mathrm{canon}\{t,\ \mathrm{plan},\ \mathrm{model},\ A,\ B,\ p_A,\ \nu_t\}), \qquad \nu_t:\ 128 \text{ random bits} \tag{9}$$

Only h_t and the screen order are returned; p_A is never sent before the choice. The random nonce ν_t prevents recovering p_A by hashing candidate values, so the commitment is hiding; the collision resistance of SHA-256 [30] makes it binding, so the server cannot later substitute a different prediction. The reveal endpoint works only after the choice has reached the server and returns the payload, so anyone can check that its hash equals the seal shown before the choice.

7.4 Presentation, repeats and reveal

For first presentations, the left side shows A when a pseudo-random draw seeded by the plan seed and trial index falls below 0.5; the participant sees *Left* and *Right*, never A and B. After all first presentations, a server-seeded sample of pairs is shown again with sides swapped, as separate trials, to measure the participant's own consistency. Answers are ordinary preference records with mode = evaluation, the trial id, the locked model version and the side shown on the left; they are never training-eligible. Unity reveals nothing during the test: showing each prediction straight after each choice would let the participant learn what the model expects and drift towards it, inflating accuracy. Only when the participant presses **Finish** does Unity request every reveal, re-hash each payload, check it against its seal, trial and pair, and close the plan (Fig. 8).

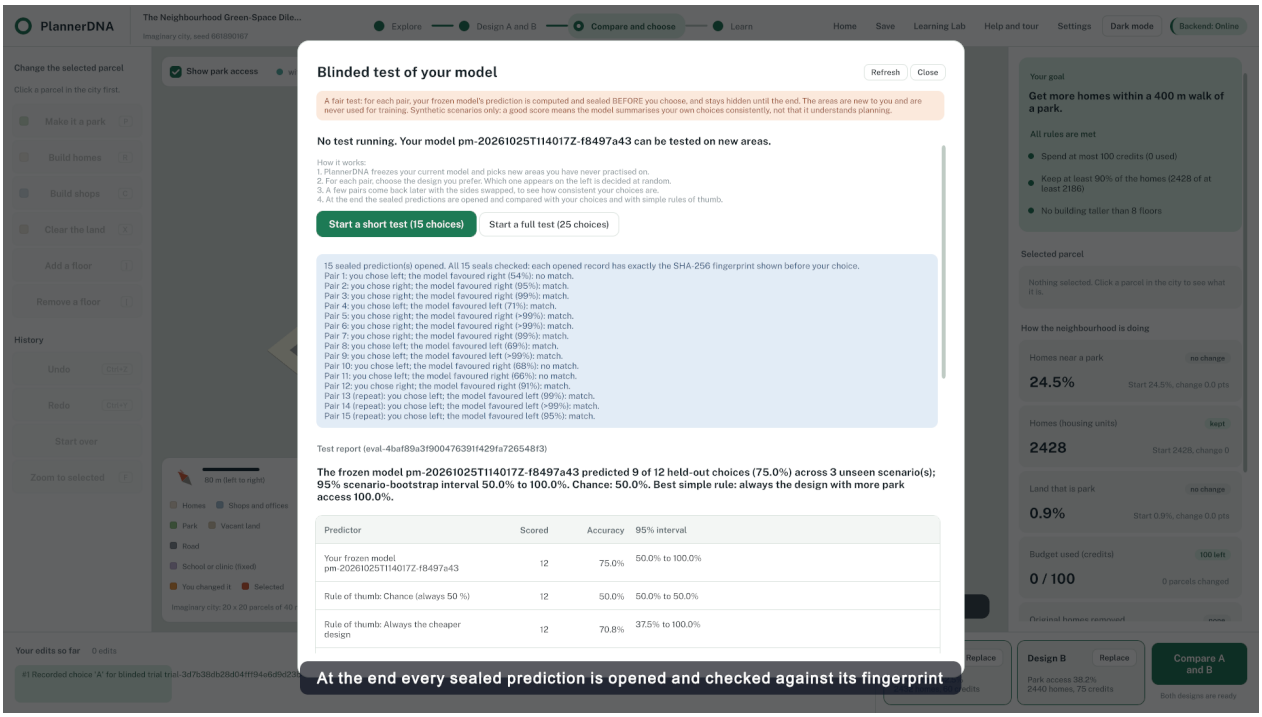


Fig. 8. End of a blinded test in the client: every sealed prediction has been opened and checked against its fingerprint before the report is shown. (Frame from the scripted demonstration run, not the case-study test.)

7.5 Scoring and the self-verifying report

Only first presentations answered A or B are scored; ties, skips, unanswered trials and repeats are counted but not scored, so the denominator is always stated. The report gives accuracy, Brier score and log loss for the model, and the same measures for chance, cheaper, more park access, equal weights and *always the left design*, which detects a side habit, each with the model-minus-baseline accuracy difference. Intervals come from 1000 bootstrap resamples of whole scenarios (seed 20261025) with 95% percentile limits, omitted when fewer than three scenarios are scored. Repeat consistency is reported with a Wilson score interval [39, 40]:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{10}$$

The share of left choices is tested against 50% with an exact two-sided binomial test, and calibration bins the probability of the favoured design (0.5 to 1.0 in steps of 0.1) and reports the expected calibration error [29]:

$$\mathrm{ECE} = \sum_b \frac{n_b}{n} |\bar{c}_b - \bar{o}_b| \tag{11}$$

where, for bin b with n_b choices, the two terms are the mean confidence and the observed rate at which the favoured design was chosen. A held-out guarantee recomputes the number of evaluation choices and evaluation scenarios present in the frozen model's training data (zero by construction). The report is computed once at closing, stored as canonical JSON with its SHA-256, and on every read the server both re-hashes the stored report and recomputes it from the stored rows; it is marked reproducible only when all three hashes agree. Warnings are generated for small samples, fewer than five scenarios, intervals that include 50%, a baseline at least as accurate as the model, low consistency and side bias.

8 Implementation and verification

Table 5 summarises the implementation. Every version identifier (generator, metrics, cost rules, features, schema, ML and protocol) is written into each stored record, so any number can be traced to the rules that produced it.

Table 5. Implementation stack.

Component	Technology
Client	Unity 6000.6.7f1, Universal Render Pipeline, UI Toolkit, Input System; C#
Server	Python 3.12.15, FastAPI, Pydantic, SQLite (migrations, foreign keys, BEGIN IMMEDIATE)
Model	scikit-learn 1.6.1, NumPy 2.2.1, SciPy 1.18.1
Figures	matplotlib [41] from export files; one CSV per figure; manifest with refit check
Determinism	SplitMix64 generation; FNV-1a content hashes; SHA-256 payloads, models and reports
Network	loopback only (127.0.0.1); no cloud services, no telemetry

Verification is automated (Table 6). Tests that need a server start a throwaway backend on a separate port with a temporary data directory, so no test ever touches the participant's database. Reproducibility follows the recommendations of [42]: every figure is generated from an export file, with the plotted numbers in a CSV beside it, and the figure manifest refits the active model from the export and records the largest absolute weight difference (0 for the case-study model).

Table 6. Automated test suites at the time of writing.

Suite	Tests	Main properties covered
Backend (pytest)	88	recomputation and tamper refusal; idempotency; readiness refusals; symmetry; LOSO isolation; artifact checksums; candidate verification; evaluation protocol; sealed export; figures
Unity EditMode	130	generator reference values; metrics; walk distances; cost table; validator; undo/redo; candidate generator; evaluation pair rule; local store and outbox
Unity PlayMode	24 (+5 opt-in)	offline play; live sync against a throwaway backend; Learning Lab training; a complete blinded test with all seals verified and a reproducible report

9 Case study

9.1 Participant and data

The case study involves a single participant, the author, working with synthetic cities. The active model (pm-20261025T114017Z-f8497a43) was trained on 30 unique comparisons across 5 scenarios (Table 7). **10 of these were made by the participant** in the demonstration city (seed 42). **The remaining 20 were generated by a scripted agent following a fixed rule**, “prefer more park access, then lower cost”, in four further cities, to reach the five scenarios recommended before a model is called non-exploratory. They are flagged in the data, and every figure states that 35 of the 61 stored choices were rule-made. This composition matters for every result below.

Table 7. Training data of the active model (A chosen 16, B chosen 14; no comparison ignored as uninformative).

Scenario	Comparisons	Source
green-42	10	participant
green-123	5	fixed rule
green-2024	5	fixed rule
green-7	5	fixed rule
green-777	5	fixed rule
Total	30	10 participant, 20 rule

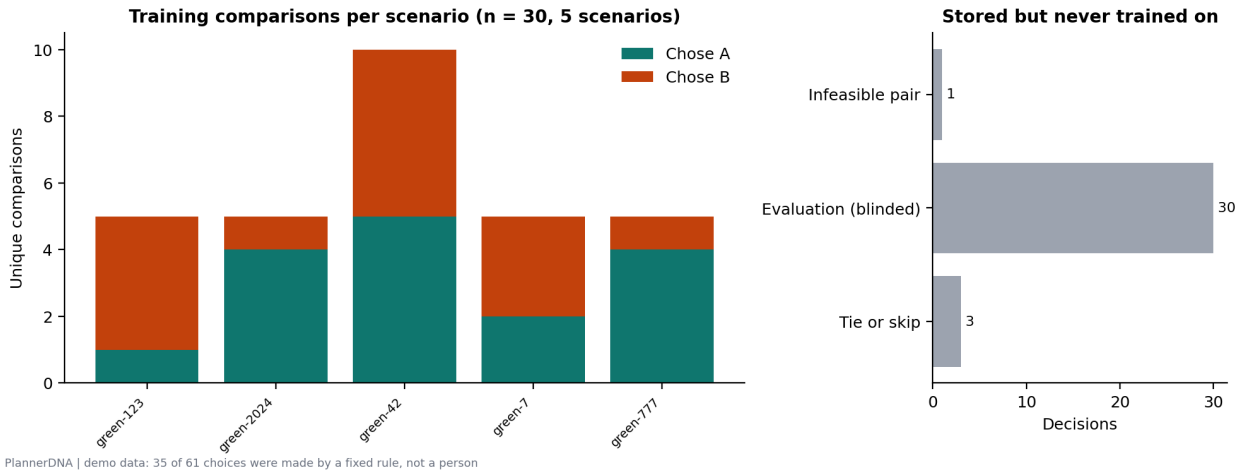


Fig. 9. Training comparisons per scenario (left) and stored decisions that were never trained on (right): one infeasible pair, the answers from two blinded tests and three ties.

9.2 Fitted model

Table 8 and Fig. 10 show the weights. Park access dominates (standardised weight 2.69, 90% interval 2.13 to 2.93, same sign in all resamples). Housing capacity and low spending carry negative weights: chosen designs tended to have less housing and higher cost. Park share is near zero and the direction of the original-housing weight is uncertain.

Table 8. Weights of the active model with 90% scenario-bootstrap intervals (200 resamples).

Feature	Standardised	90% interval	Same sign	Raw
Park access	+2.69	[+2.13, +2.93]	100%	+18.14
Housing capacity	-0.60	[-1.17, -0.05]	95%	-83.79
Park share of land	+0.10	[-0.43, +1.10]	63%	+16.97
Low spending	-0.79	[-0.99, -0.48]	100%	-3.04
Keeping original housing	+0.47	[-0.18, +0.97]	87%	+92.56

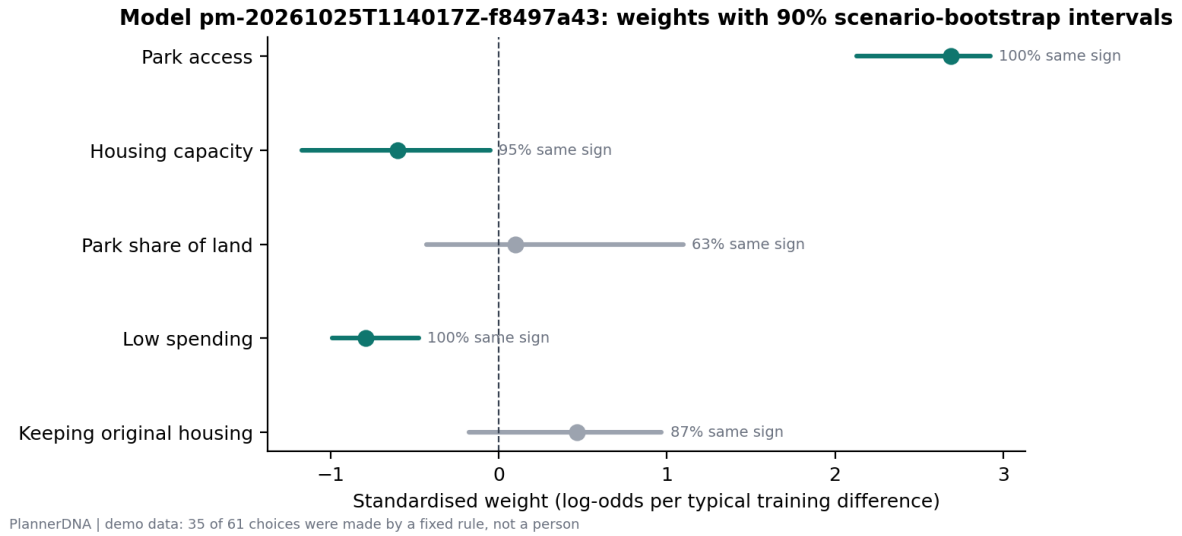


Fig. 10. Standardised weights with 90% scenario-bootstrap intervals and sign agreement. Grey intervals include zero.

Leave-one-scenario-out accuracy was 96.7% (Brier 0.069, log loss 0.241), identical in accuracy to the rule "always more park access" (96.7%, Brier 0.033) and well above chance (50.0%), the equal-weight rule (50.0%) and the cheaper-design rule (36.7%) (Fig. 11). Training-set accuracy was 96.7%. The learning curve (Fig. 12) shows the model approaching, but never exceeding, the more-access rule as comparisons

accumulate.

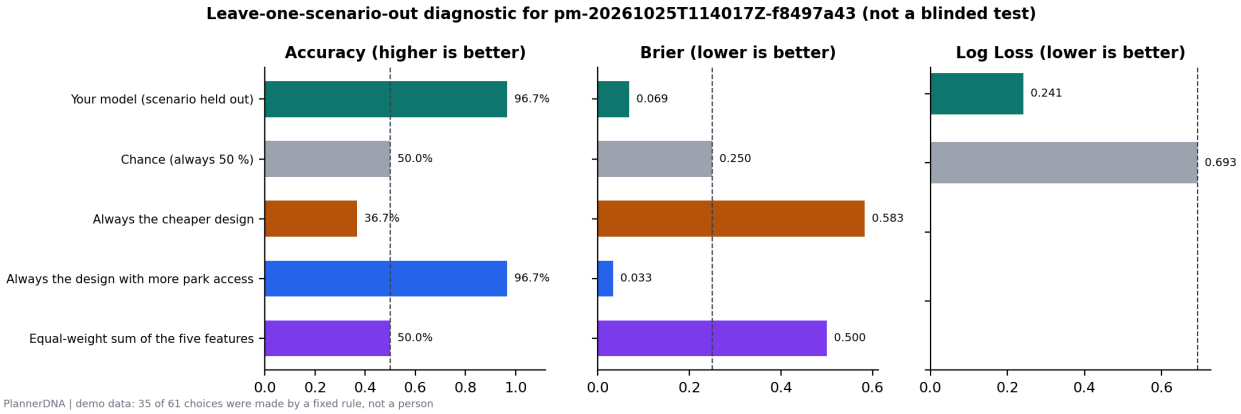


Fig. 11. Leave-one-scenario-out diagnostic of the active model against four rules (inside the training data; not a held-out test).

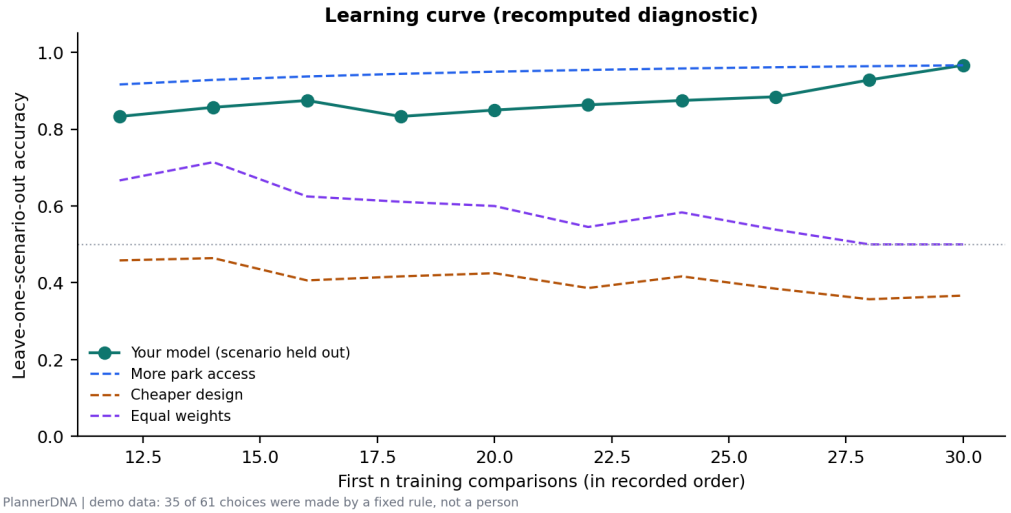


Fig. 12. Recomputed learning curve: LOSO accuracy of models fitted on the first n comparisons, with the rules evaluated on the same comparisons.

9.3 Blinded held-out test

The participant ran one blinded test (plan eval-11f5b0d7657a45d0b4b62cf05e9c6b26): 4 pairs in each of 3 unseen scenarios plus 3 repeated pairs, all 15 answered. 3 first presentations were answered as ties, leaving 9 scored choices. All 15 seals were opened and verified, the held-out guarantee recomputed zero evaluation choices and scenarios in the training data, and the stored report re-hashed and recomputed to the same SHA-256 (reproducible = true).

Table 9. Blinded held-out results on the participant's scored choices (95% scenario-bootstrap intervals, 1000 resamples). The last column is model minus rule accuracy in percentage points.

Predictor	Correct	Accuracy	95% interval	Brier	Log loss	Model − rule
Frozen model	8 / 9	88.9%	[50%, 100%]	0.111	0.661	–
Always the design with more park access	8 / 9	88.9%	[50%, 100%]	0.111	–	+0.0 [+0, +0]
Always the design shown on the left	6 / 9	66.7%	[50%, 100%]	0.333	–	+22.2 [-50, +50]
Chance (0.5)	4.5 / 9	50.0%	[50%, 50%]	0.250	0.693	+38.9 [+0, +50]
Equal-weight sum of the five features	4 / 9	44.4%	[25%, 67%]	0.556	–	+44.4 [+0, +75]
Always the cheaper design	2 / 9	22.2%	[0%, 50%]	0.778	–	+66.7 [+0, +100]

The frozen model predicted 8 of 9 choices (88.9%, 95% interval 50.0% to 100.0%; Table 9, Fig. 13). The rule “always more park access” achieved exactly the same, and the two made the same prediction on 9 of the 9 scored pairs. Per scenario, the model was correct on 4 of 4, 3 of 3, 1 of 2 scored choices. All predictions were confident (favoured-design probability at least 0.94), so every choice fell in the top calibration bin, where the mean confidence was 0.976 against an observed rate of 0.889 (ECE 0.088). The single error was made with high confidence (Fig. 15), which is what drives the log loss (0.661) above that of chance (0.693) even though the Brier score (0.111) is far below chance (0.250).

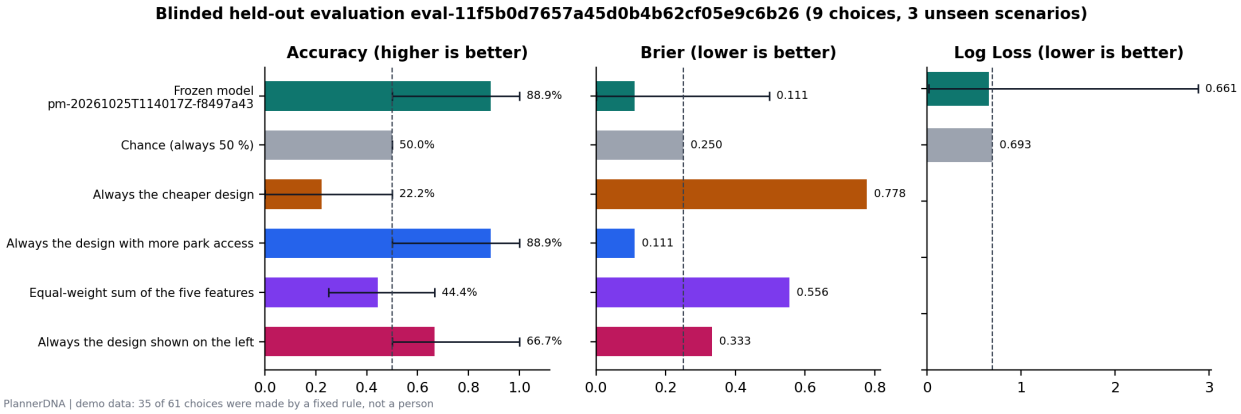


Fig. 13. Blinded held-out accuracy, Brier score and log loss of the frozen model against chance and four rules, with 95% scenario-bootstrap intervals.

The participant chose the same design in 2 of 3 repeated pairs (67%, Wilson 95% interval 21% to 94%), and chose the left design in 7 of 12 binary choices (two-sided binomial p = 0.77), giving no evidence of a side habit (Fig. 14). Because the frozen model assigns both presentations of a repeated pair the same probability, its own repeat consistency is 100% by construction; the participant's consistency is an approximate ceiling on how well any model of these choices can do.

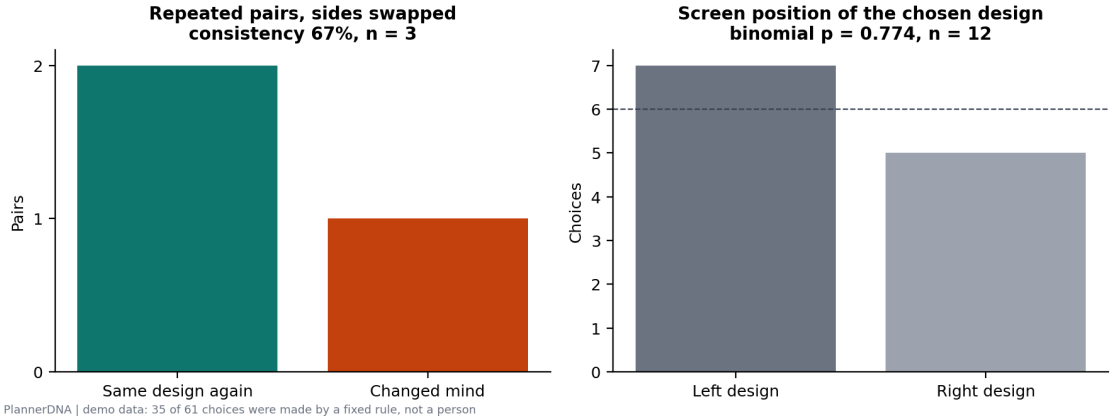


Fig. 14. Repeat consistency (left) and the screen position of the chosen design (right) in the participant's blinded test.

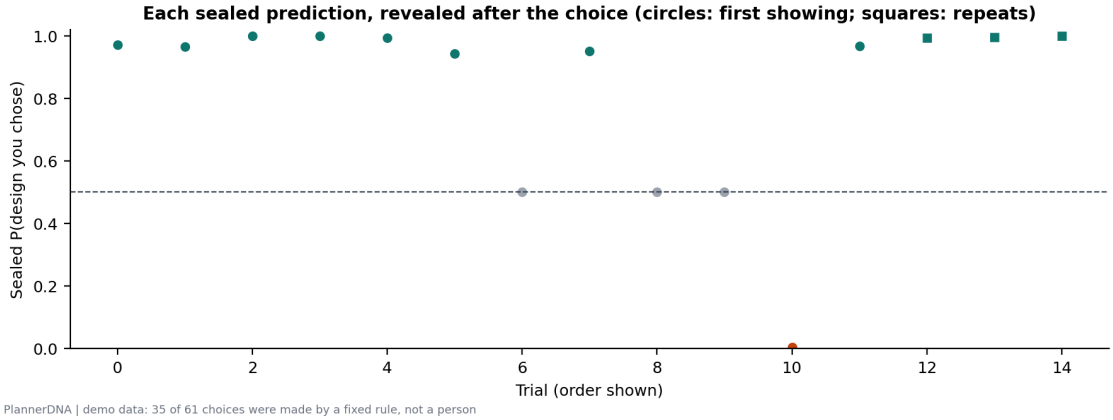


Fig. 15. Every sealed prediction, revealed after the choice: the probability the frozen model gave the design the participant chose, in the order shown (circles: first presentations; squares: repeats; grey: ties).

A second blinded test was answered by the scripted agent following the same rule that generated the 20 rule-made training comparisons. The model predicted 12 of 12 scored choices, and so did the more-access rule (100.0%). This test mainly confirms that the pipeline works end to end; it is not evidence about a person.

10 Discussion

The model has not shown more than a one-line rule. On the participant's held-out choices the frozen model and the rule “always choose more park access” were equally accurate and made the same prediction on every scored pair. With no discordant pairs, the data carry no information about which of the two is better, in the sense of McNemar's test for paired proportions [43]. This is the expected outcome when one attribute dominates the choices that generated the training data, and two-thirds of those choices were generated by a rule whose first criterion was exactly that attribute. A linear model then collapses towards the dominant attribute; to show that it has learned more, a test needs pairs in which park access conflicts with the other objectives strongly enough that a person might give it up. The model-blind trade-off rule guarantees conflicting pairs, but not that the conflicts are large.

Weights are associations, not motives. The negative weights on low spending and housing capacity do not mean that the participant likes spending or dislikes housing. Creating a park costs credits and often replaces buildings, so in the compared pairs more access usually came with higher cost and less housing (see the chosen-minus-rejected differences in Fig. 16). A model that tracks access inherits those correlations as weights. The Learning Lab therefore phrases every weight as “your choices were associated with …” and never displays an exchange rate built from a weight whose interval includes zero.

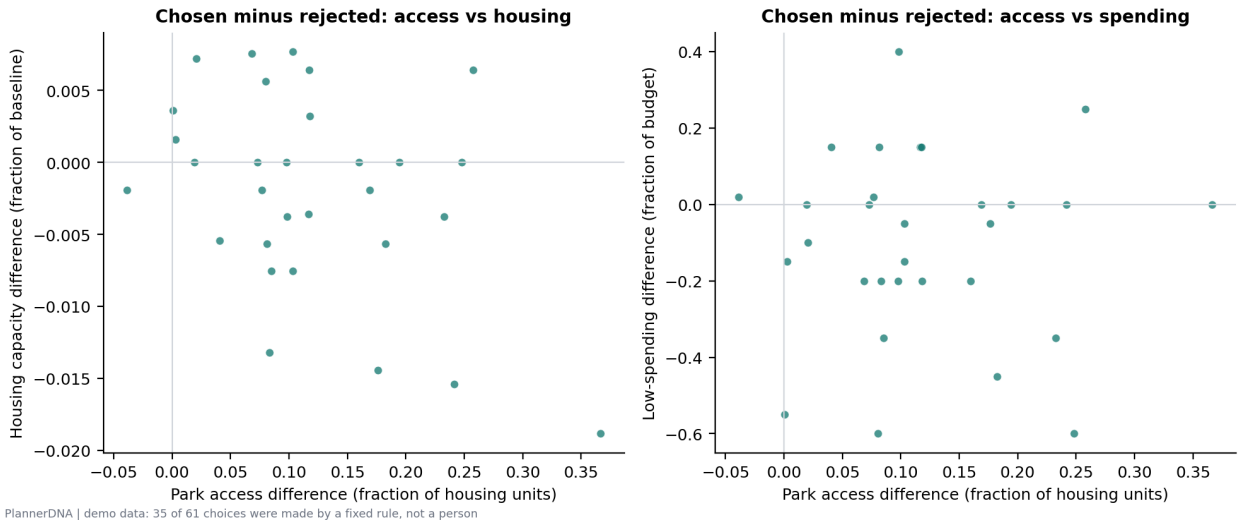


Fig. 16. Chosen minus rejected design for every training comparison: park access against housing capacity (left) and against low spending (right). Chosen designs almost always had more access, often at higher cost.

Small samples give wide intervals. With nine scored choices, one choice moves accuracy by 11.1 percentage points, and the 95% interval runs from 50% to 100%, so the test cannot distinguish the model from guessing at conventional confidence. A percentile bootstrap over only three scenarios is, moreover, rough and can be optimistic. As a rough planning figure, a normal approximation suggests the number of scored choices needed for a given interval half-width h:

$$n \approx \frac{z^2\, p(1-p)}{h^2} = \frac{1.96^2 \times 0.85 \times 0.15}{0.10^2} \approx 49 \tag{12}$$

that is, about 50 scored choices for a half-width of 10 points around an accuracy of 85%, and many more discordant pairs than the zero observed here before the model could be compared with the rule.

What the protocol does provide. Within these limits, the result is trustworthy in a specific sense: the model could not have been trained on the test cities or choices (checked by id, seed and mode, and recomputed in the report); the predictions demonstrably existed before the choices (seals verified); the test pairs were not chosen by or for the model; neither the participant nor the client chose the screen sides; the participant saw no prediction during the test; and the reported numbers can be recomputed from the stored rows by anyone with the export. These properties are enforced by refusals in the server, which makes them testable, and they were tested. The same protocol can evaluate future models (for example, sequence models of the editing process) on equal terms.

Teaching use. For students, the value lies less in the model than in the loop: designing two complete alternatives under explicit constraints, seeing every metric recomputed transparently, committing to a choice, and then confronting an honest test of whether a simple summary of one's choices predicts the next ones, including the possibility that one's choices are less consistent than expected.

11 Limitations and ethical considerations

- **One participant, who is also the developer.** Nothing here generalises to other people, and the participant's familiarity with the metrics may make choices more rule-like than a student's would be.
- **Rule-generated training data.** 20 of the 30 training comparisons were generated by a fixed rule. The fitted weights largely reflect that rule, and the agreement between model and rule in the participant's test is partly inherited from it.
- **Synthetic cities and proxy metrics.** Housing units are a capacity proxy, credits are fictional and access is a grid-road distance. The system knows nothing about real places, residents, land values or equity, and its outputs are not planning advice.

- **Stated, hypothetical choices.** Choices between on-screen designs need not reflect what a person would support in a real decision with real stakes.
- **Small evaluation.** Nine scored choices in three scenarios; intervals are wide and the scenario bootstrap with three clusters is crude.
- **Linear model on five features.** Interactions, thresholds (for example, caring about access only below some level) and the reasons participants type are not modelled.
- **Demonstration video.** The accompanying video was recorded on a copy of the data with scripted choices, and says so; the case-study numbers in this paper come only from the participant's own test.

All data are synthetic or produced by the author on a single computer; no data about other people were collected, and nothing leaves the machine. The client offers export and complete deletion of local and server data. If the system were used with students, participation, data retention and the framing of model outputs as associations rather than judgments would need appropriate consent and review.

12 Conclusion

PlannerDNA makes the trade-offs of a small planning exercise explicit, records which ones a person chooses, summarises those choices with a model that can be read line by line, and tests that summary under a protocol in which the important safeguards (frozen model, unseen scenarios, sealed predictions, server-held randomisation, model-blind pairs, delayed reveal and a self-verifying report) are enforced by the software. In the case study the model predicted 8 of 9 held-out choices, exactly as many as the rule “always choose more park access”, and the system reports that plainly. We regard that sentence, rather than the accuracy figure, as the main result: a workflow that keeps a model honest is worth more than a model that looks good.

Data and software availability

All records are stored locally and can be exported as a single JSON file containing scenarios, sessions, events, alternatives, comparisons, model runs, predictions, evaluation plans and trials. Figures are regenerated from an export with backend/tools/make_figures.py, which writes the CSV behind each figure and a manifest with a refit check. Model files are JSON. The scripts that produced the demonstration video, the carousel and this paper are in the tools folder of the project.

Use of AI assistance

Software development and drafting of this report were assisted by an AI coding assistant. All numbers reported here were produced by the system's own code from the stored data and are pulled from the local server when the paper is built.

References

[1] WHO Regional Office for Europe, *Urban Green Spaces and Health: A Review of Evidence*. Copenhagen, Denmark: World Health Organization, 2016.

[2] S. Geertman and J. Stillwell, “Planning support systems: An inventory of current practice,” *Computers, Environment and Urban Systems*, vol. 28, no. 4, pp. 291–310, 2004.

[3] A. Poplin, “Playful public participation in urban planning: A case study for online serious games,” *Computers, Environment and Urban Systems*, vol. 36, no. 3, pp. 195–206, 2012.

[4] L. L. Thurstone, “A law of comparative judgment,” *Psychological Review*, vol. 34, no. 4, pp. 273–286, 1927.

[5] R. A. Bradley and M. E. Terry, “Rank analysis of incomplete block designs: I. The method of paired comparisons,” *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.

[6] R. D. Luce, *Individual Choice Behavior: A Theoretical Analysis*. New York, NY, USA: Wiley, 1959.

[7] D. McFadden, “Conditional logit analysis of qualitative choice behavior,” in *Frontiers in Econometrics*, P. Zarembka, Ed. New York, NY, USA: Academic Press, 1974, pp. 105–142.

[8] R. Herbrich, T. Graepel, and K. Obermayer, “Large margin rank boundaries for ordinal regression,” in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 2000, pp. 115–132.

[9] T. Joachims, “Optimizing search engines using clickthrough data,” in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002, pp. 133–142.

[10] J. Fürnkranz and E. Hüllermeier, Eds., *Preference Learning*. Berlin, Germany: Springer, 2010.

[11] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4299–4307.

[12] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, art. 15, 2012.

[13] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, art. 100804, 2023.

[14] A. Tversky and D. Kahneman, “Judgment under uncertainty: Heuristics and biases,” *Science*, vol. 185, no. 4157, pp. 1124–1131, 1974.

[15] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, “The preregistration revolution,” *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.

[16] L. N. Oliver, N. Schuurman, and A. W. Hall, “Comparing circular and network buffers to examine the influence of land use on walking for leisure and errands,” *International Journal of Health Geographics*, vol. 6, art. 41, 2007.

[17] J. J. Louviere, D. A. Hensher, and J. D. Swait, *Stated Choice Methods: Analysis and Applications*. Cambridge, U.K.: Cambridge University Press, 2000.

[18] P. A. Samuelson, “A note on the pure theory of consumer’s behaviour,” *Economica*, vol. 5, no. 17, pp. 61–71, 1938.

[19] A. Tversky, “Intransitivity of preferences,” *Psychological Review*, vol. 76, no. 1, pp. 31–48, 1969.

[20] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[21] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[22] D. R. Roberts *et al.*, “Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure,” *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.

[23] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[24] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[25] C. A. Field and A. H. Welsh, “Bootstrapping clustered data,” *Journal of the Royal Statistical Society: Series B*, vol. 69, no. 3, pp. 369–390, 2007.

[26] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[27] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[28] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632.

[29] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, “Obtaining well calibrated probabilities using Bayesian binning,” in *Proc. 29th AAAI Conf. Artificial Intelligence*, 2015, pp. 2901–2907.

[30] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Gaithersburg, MD, USA, Aug. 2015.

[31] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178.

[32] Unity Technologies, “Unity 6 real-time development platform,” version 6000.6.7f1, software, 2026. [Online]. Available: https://unity.com

[33] S. Ramírez, “FastAPI,” software. [Online]. Available: https://fastapi.tiangolo.com

[34] D. R. Hipp *et al.*, “SQLite,” software. [Online]. Available: https://www.sqlite.org

[35] G. L. Steele Jr., D. Lea, and C. H. Flood, “Fast splittable pseudorandom number generators,” in *Proc. ACM Int. Conf. Object Oriented Programming Systems Languages & Applications (OOPSLA)*, 2014, pp. 453–472.

[36] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[37] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[38] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.

[39] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[40] A. Agresti and B. A. Coull, “Approximate is better than ‘exact’ for interval estimation of binomial proportions,” *The American Statistician*, vol. 52, no. 2, pp. 119–126, 1998.

[41] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[42] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, “Ten simple rules for reproducible computational research,” *PLoS Computational Biology*, vol. 9, no. 10, art. e1003285, 2013.

[43] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.